```
# Name : Prathamesh Lavange
# div : B
# Roll No :31
# PRN : 72332154L
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
import pandas as pd

url = "https://raw.githubusercontent.com/subashgandyer/datasets/main/Real%20estate.csv"

df = pd.read_csv(url)

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)

df.head()
```

```
Dataset loaded successfully!
Dataset shape: (414, 8)
```

	No	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
0	1	2012.917	32.0	84.87882	10	24.98298	121.54024	37.9
1	2	2012.917	19.5	306.59470	9	24.98034	121.53951	42.2
2	3	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3
3	4	2013.500	13.3	561.98450	5	24.98746	121.54391	54.8
4	5	2012.833	5.0	390.56840	5	24.97937	121.54245	43.1

Next steps: Generate code with df | New interactive sheet

```
print("Dataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns)
```

```
print("\nDataset Information:")
df.info()

print("\nStatistical Summary:")
df.describe()
```

```
Dataset Shape:
(414, 8)

Column Names:
Index(['No', 'X1 transaction date', 'X2 house age',
       'X3 distance to the nearest MRT station',
       'X4 number of convenience stores', 'X5 latitude', 'X6 longitude',
       'Y house price of unit area'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 414 entries, 0 to 413
Data columns (total 8 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   No                                      414 non-null    int64
 1   X1 transaction date                     414 non-null    float64
 2   X2 house age                            414 non-null    float64
 3   X3 distance to the nearest MRT station  414 non-null    float64
 4   X4 number of convenience stores         414 non-null    int64
 5   X5 latitude                             414 non-null    float64
 6   X6 longitude                            414 non-null    float64
 7   Y house price of unit area              414 non-null    float64
dtypes: float64(6), int64(2)
memory usage: 26.0 KB

Statistical Summary:
```

	No	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
count	414.000000	414.000000	414.000000	414.000000	414.000000	414.000000	414.000000	414.000000
mean	207.500000	2013.148971	17.712560	1083.885689	4.094203	24.969030	121.533361	37.980193
std	119.655756	0.281967	11.392485	1262.109595	2.945562	0.012410	0.015347	13.606488
min	1.000000	2012.667000	0.000000	23.382840	0.000000	24.932070	121.473530	7.600000
25%	104.250000	2012.917000	9.025000	289.324800	1.000000	24.963000	121.528085	27.700000
50%	207.500000	2013.167000	16.100000	492.231300	4.000000	24.971100	121.538630	38.450000
75%	310.750000	2013.417000	28.150000	1454.279000	6.000000	24.977455	121.543305	46.600000
max	414.000000	2013.583000	43.800000	6488.021000	10.000000	25.014590	121.566270	117.500000

```
df.columns = [
    "property_id",
    "transaction_date",
    "house_age",
    "distance_to_mrt",
    "convenience_stores",
    "latitude",
    "longitude",
    "house_price_per_unit"
]

print("Columns renamed successfully!")
print(df.columns)

df.head()
```

```
Columns renamed successfully!
Index(['property_id', 'transaction_date', 'house_age', 'distance_to_mrt',
       'convenience_stores', 'latitude', 'longitude', 'house_price_per_unit'],
      dtype='object')
```

	property_id	transaction_date	house_age	distance_to_mrt	convenience_stores	latitude	longitude	house_price_per_unit
0	1	2012.917	32.0	84.87882	10	24.98298	121.54024	37.9
1	2	2012.917	19.5	306.59470	9	24.98034	121.53951	42.2
2	3	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3
3	4	2013.500	13.3	561.98450	5	24.98746	121.54391	54.8
4	5	2012.833	5.0	390.56840	5	24.97937	121.54245	43.1

Next steps: Generate code with df | New interactive sheet

```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
property_id             0
transaction_date        0
house_age               0
distance_to_mrt         0
convenience_stores      0
latitude                0
longitude               0
house_price_per_unit    0
dtype: int64
```

```
# Fill missing numerical values with median
```

```
for col in df.select_dtypes(include=np.number).columns:
    df[col] = df[col].fillna(df[col].median())

print("Missing values handled successfully!")

print(df.isnull().sum())
```

```
Missing values handled successfully!
property_id             0
transaction_date        0
house_age               0
distance_to_mrt         0
convenience_stores      0
latitude                0
longitude               0
house_price_per_unit    0
dtype: int64
```

```
# Houses less than 10 years old
young_houses = df[df["house_age"] < 10]

print("Houses less than 10 years old:")
print(young_houses.head())

print("Total young houses:", len(young_houses))


# Houses within 1000 meters of MRT station
near_mrt = df[df["distance_to_mrt"] <= 1000]

print("\nHouses within 1000 meters of MRT station:")
print(near_mrt.head())

print("Total houses near MRT:", len(near_mrt))
```

```
Houses less than 10 years old:
    property_id  transaction_date  house_age  distance_to_mrt  \
4             5          2012.833        5.0        390.56840
5             6          2012.667        7.1       2175.03000
11           12          2013.333        6.3         90.45606
16           17          2013.250        0.0        292.99780
19           20          2012.667        1.5         23.38284

    convenience_stores  latitude  longitude  house_price_per_unit
4                    5  24.97937  121.54245                  43.1
5                    3  24.96305  121.51254                  32.1
11                   9  24.97433  121.54310                  58.1
16                   6  24.97744  121.54458                  70.1
19                   7  24.96772  121.54102                  47.7
Total young houses: 109
```

```
Houses within 1000 meters of MRT station:
   property_id  transaction_date  house_age  distance_to_mrt  \
0            1          2012.917       32.0         84.87882
1            2          2012.917       19.5        306.59470
2            3          2013.583       13.3        561.98450
3            4          2013.500       13.3        561.98450
4            5          2012.833        5.0        390.56840

   convenience_stores  latitude  longitude  house_price_per_unit
0                  10  24.98298  121.54024                  37.9
1                   9  24.98034  121.53951                  42.2
2                   5  24.98746  121.54391                  47.3
3                   5  24.98746  121.54391                  54.8
4                   5  24.97937  121.54245                  43.1
Total houses near MRT: 276
```

```
# Select important columns
df_subset = df[
    [
        "house_age",
        "distance_to_mrt",
        "convenience_stores",
        "house_price_per_unit"
    ]
]

print("Selected columns:")
display(df_subset.head())

print("\nSubset shape:", df_subset.shape)
```

```
Selected columns:
```

	house_age	distance_to_mrt	convenience_stores	house_price_per_unit
0	32.0	84.87882	10	37.9
1	19.5	306.59470	9	42.2
2	13.3	561.98450	5	47.3
3	13.3	561.98450	5	54.8
4	5.0	390.56840	5	43.1

```
Subset shape: (414, 4)
```

```
# Create age groups
bins = [0, 10, 20, 30, 40, np.inf]
labels = ["0-10", "10-20", "20-30", "30-40", "40+"]
```

```
df["age_group"] = pd.cut(
    df["house_age"],
    bins=bins,
    labels=labels
)

print("Age groups created successfully!")

display(df[["house_age", "age_group"]].head(10))
```

```
Age groups created successfully!
```

	house_age	age_group
0	32.0	30-40
1	19.5	10-20
2	13.3	10-20
3	13.3	10-20
4	5.0	0-10
5	7.1	0-10
6	34.5	30-40
7	20.3	20-30
8	31.7	30-40
9	17.9	10-20

```
# Calculate average house price for each age group
avg_price_by_age = df.groupby(
    "age_group",
    observed=False
)["house_price_per_unit"].mean()

print("Average House Price by Age Group:")
print(avg_price_by_age)
```

```
Average House Price by Age Group:
age_group
0-10     45.232258
10-20    34.138323
20-30    31.195349
30-40    36.794186
40+      48.762500
Name: house_price_per_unit, dtype: float64
```

```
# Average house price based on number of convenience stores
avg_price_by_stores = df.groupby(
    "convenience_stores"
)["house_price_per_unit"].mean()

print("Average House Price by Convenience Stores:")
print(avg_price_by_stores)
```

```
Average House Price by Convenience Stores:
convenience_stores
0     26.462687
1     31.839130
2     31.412500
3     29.536957
4     37.474194
5     44.729851
6     46.951351
7     43.848387
8     44.696667
9     51.732000
10    48.430000
Name: house_price_per_unit, dtype: float64
```

```
# Statistical summary of the dataset
print("Statistical Summary:")

display(df.describe())
```

Statistical Summary:

	property_id	transaction_date	house_age	distance_to_mrt	convenience_stores	latitude	longitude	house_price_per_unit
count	414.000000	414.000000	414.000000	414.000000	414.000000	414.000000	414.000000	414.000000
mean	207.500000	2013.148971	17.712560	1083.885689	4.094203	24.969030	121.533361	37.980193
std	119.655756	0.281967	11.392485	1262.109595	2.945562	0.012410	0.015347	13.606488
min	1.000000	2012.667000	0.000000	23.382840	0.000000	24.932070	121.473530	7.600000
25%	104.250000	2012.917000	9.025000	289.324800	1.000000	24.963000	121.528085	27.700000
50%	207.500000	2013.167000	16.100000	492.231300	4.000000	24.971100	121.538630	38.450000
75%	310.750000	2013.417000	28.150000	1454.279000	6.000000	24.977455	121.543305	46.600000
max	414.000000	2013.583000	43.800000	6488.021000	10.000000	25.014590	121.566270	117.500000

```
# Calculate Q1 and Q3
Q1 = df["house_price_per_unit"].quantile(0.25)
Q3 = df["house_price_per_unit"].quantile(0.75)

# Calculate IQR
```

```
# Calculate IQR
IQR = Q3 - Q1

# Calculate lower and upper limits
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

# Find outliers
outliers = df[
    (df["house_price_per_unit"] < lower_bound) |
    (df["house_price_per_unit"] > upper_bound)
]

print("\nNumber of outliers:", len(outliers))
display(outliers)
```

```
Q1: 27.7
Q3: 46.6
IQR: 18.900000000000002
Lower Bound: -0.6500000000000021
Upper Bound: 74.95

Number of outliers: 3
```

	property_id	transaction_date	house_age	distance_to_mrt	convenience_stores	latitude	longitude	house_price_per_unit	age_group
220	221	2013.333	37.2	186.5101	9	24.97703	121.54265	78.3	30-40
270	271	2013.333	10.8	252.5822	1	24.97460	121.53046	117.5	10-20
312	313	2013.583	35.4	318.5292	9	24.97071	121.54069	78.0	30-40

Next steps: Generate code with outliers | New interactive sheet

```
# Remove outliers from house prices
df_clean = df[
    (df["house_price_per_unit"] >= lower_bound) &
    (df["house_price_per_unit"] <= upper_bound)
].copy()

print("Original dataset shape:", df.shape)
print("Cleaned dataset shape:", df_clean.shape)

print("\nOutliers removed:", len(df) - len(df_clean))
```

```
display(df_clean.head())
```

```
Original dataset shape: (414, 9)
Cleaned dataset shape: (411, 9)

Outliers removed: 3
```

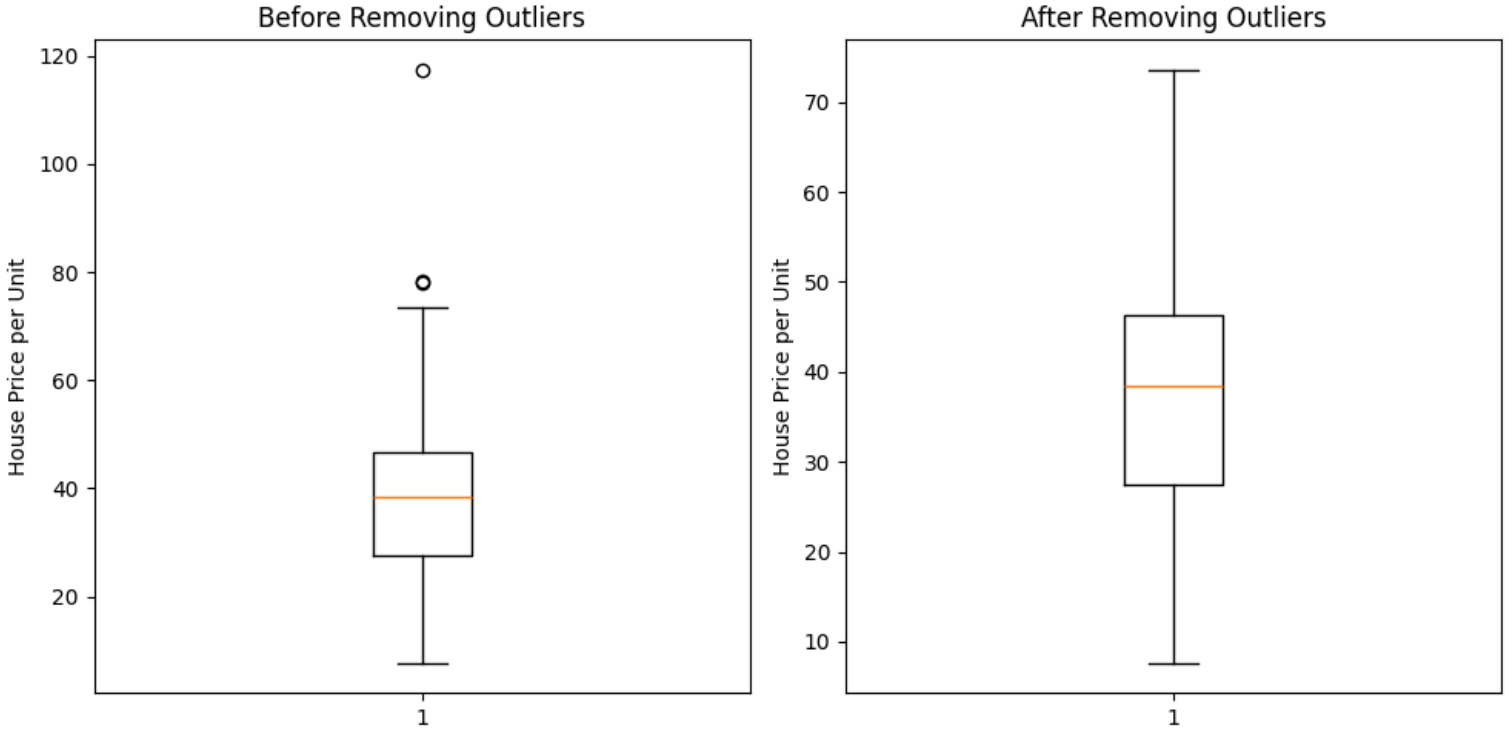
	property_id	transaction_date	house_age	distance_to_mrt	convenience_stores	latitude	longitude	house_price_per_unit	age_group
0	1	2012.917	32.0	84.87882	10	24.98298	121.54024	37.9	30-40
1	2	2012.917	19.5	306.59470	9	24.98034	121.53951	42.2	10-20
2	3	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3	10-20
3	4	2013.500	13.3	561.98450	5	24.98746	121.54391	54.8	10-20
4	5	2012.833	5.0	390.56840	5	24.97937	121.54245	43.1	0-10

```
# Create boxplots before and after removing outliers
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.boxplot(df["house_price_per_unit"].dropna())
plt.title("Before Removing Outliers")
plt.ylabel("House Price per Unit")

plt.subplot(1, 2, 2)
plt.boxplot(df_clean["house_price_per_unit"].dropna())
plt.title("After Removing Outliers")
plt.ylabel("House Price per Unit")

plt.tight_layout()
plt.show()
```




```
# Verify the cleaned dataset
print("Final Dataset Shape:", df_clean.shape)

print("\nMissing Values:")
print(df_clean.isnull().sum())

# Save cleaned dataset
df_clean.to_csv("Real_Estate_Cleaned.csv", index=False)

print("\nCleaned dataset saved successfully!")
```

```
Final Dataset Shape: (411, 9)

Missing Values:
property_id             0
transaction_date        0
house_age               0
distance_to_mrt         0
convenience_stores      0
latitude                0
longitude               0
house_price_per_unit    0
```

```
age_group               17
dtype: int64

Cleaned dataset saved successfully!
```

```
from google.colab import files

files.download("Real_Estate_Cleaned.csv")
```